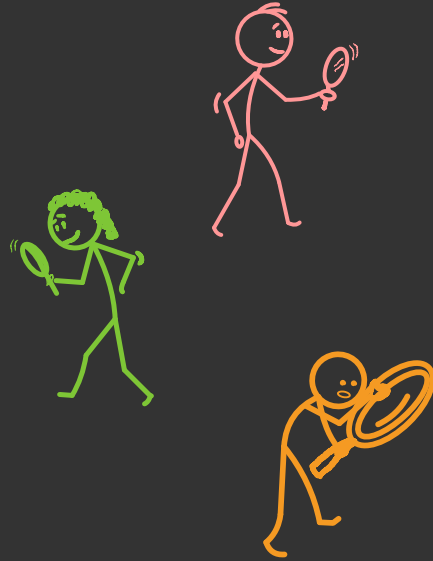


trees

CSCI
134

woodle's r&d
team continues
to scour the
globe for new
and interesting
five-letter words

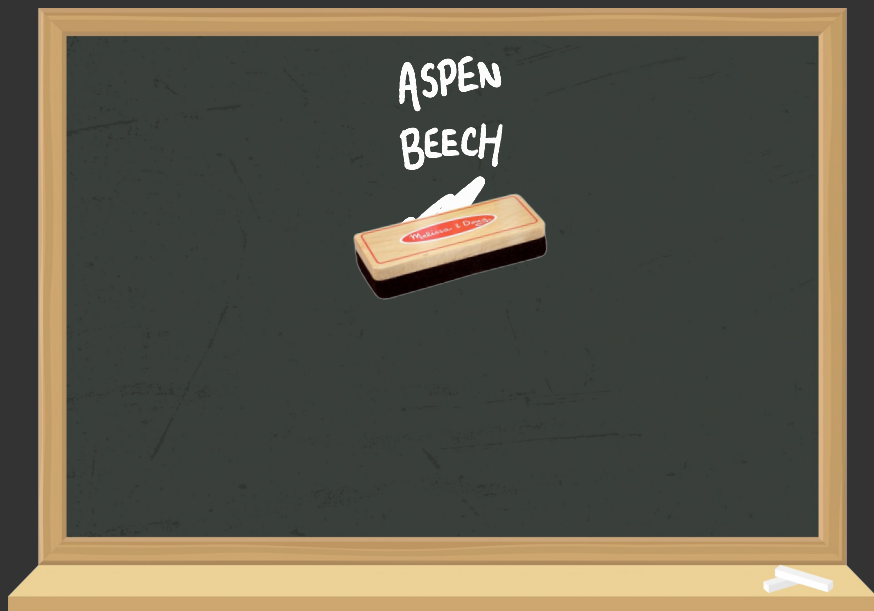




by jove,
birch!



as field reports come back,
woodle hq updates its set of woods



by jove,
birch!



to keep the list in sorted order,
we have to **erase** and **rewrite** woods
(about half of them on average)



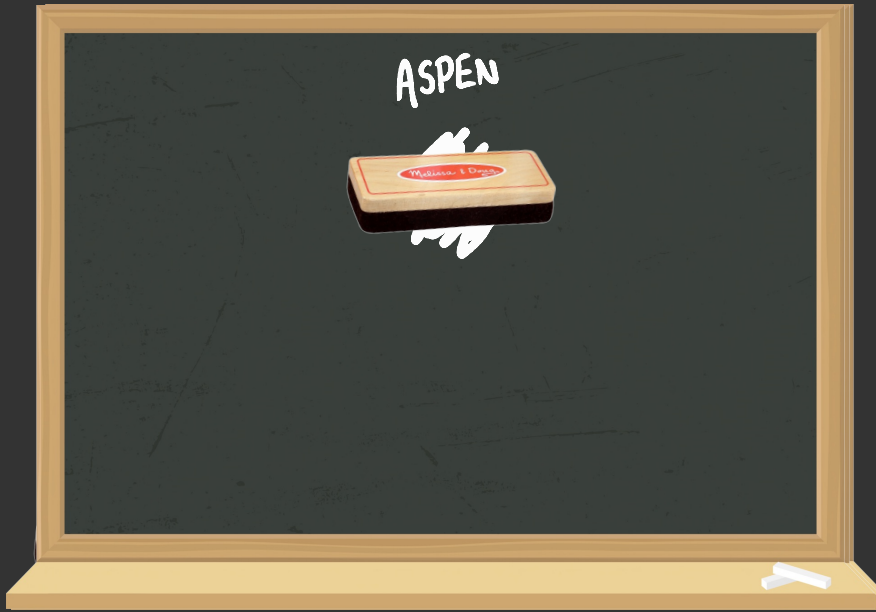
by jove,
birch!



to keep the list in sorted order,
we have to **erase** and **rewrite** woods
(about half of them on average)



to keep the list in sorted order,
we have to **erase** and **rewrite** woods
(about half of them on average)



to keep the list in sorted order,
we have to **erase** and **rewrite** words
(about half of them on average)



to keep the list in sorted order,
we have to **erase** and **rewrite** woods
(about half of them on average)

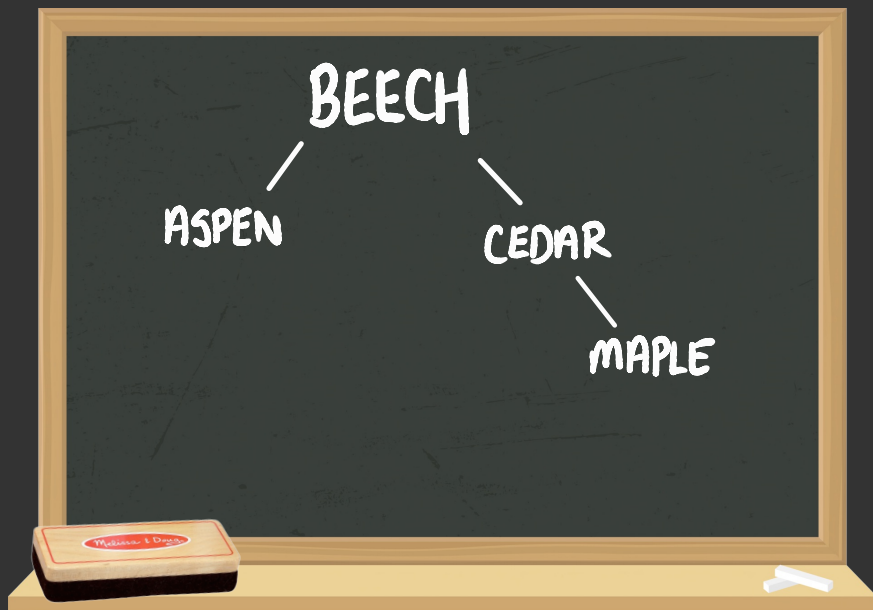


my heavens,
ebony!

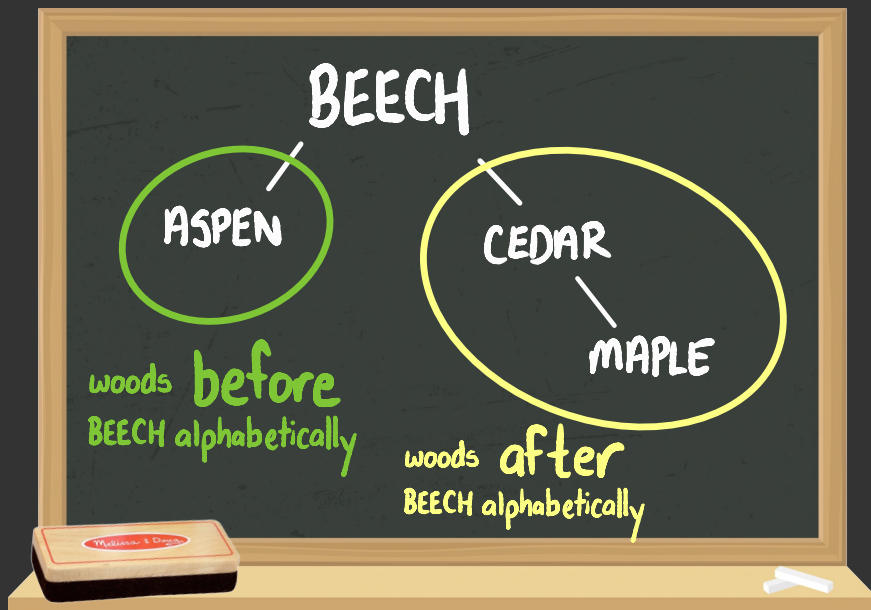


this gets **old** fast

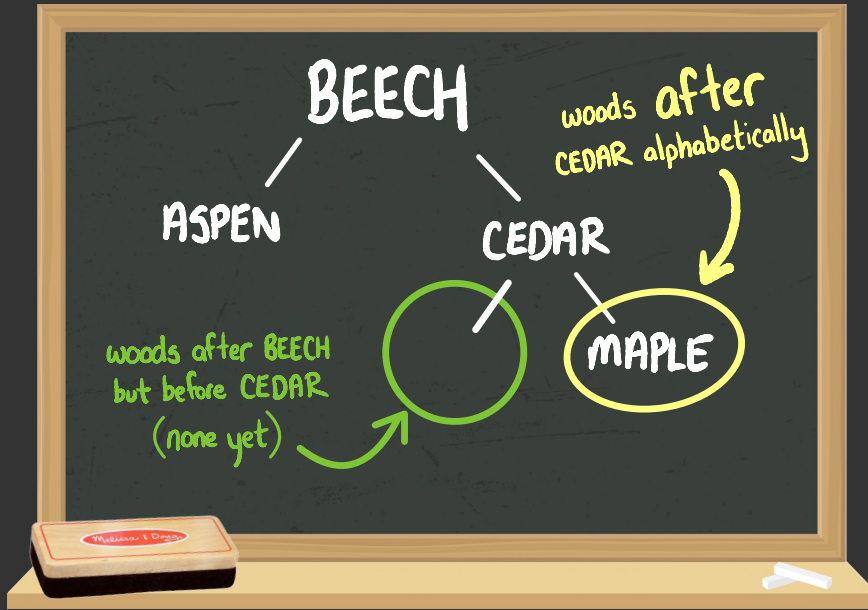
is there an easier way?



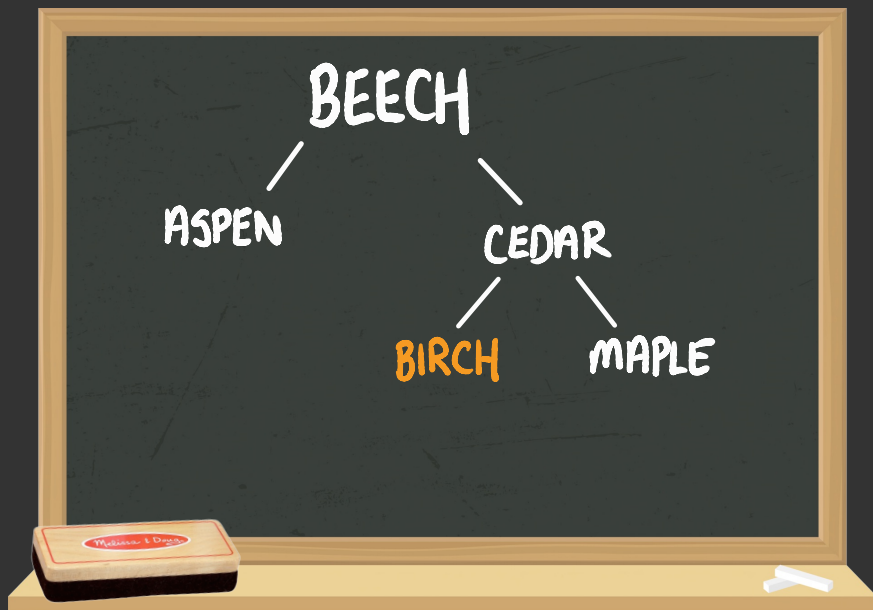
idea: organize the woods into a tree!



idea: organize the woods into a tree!



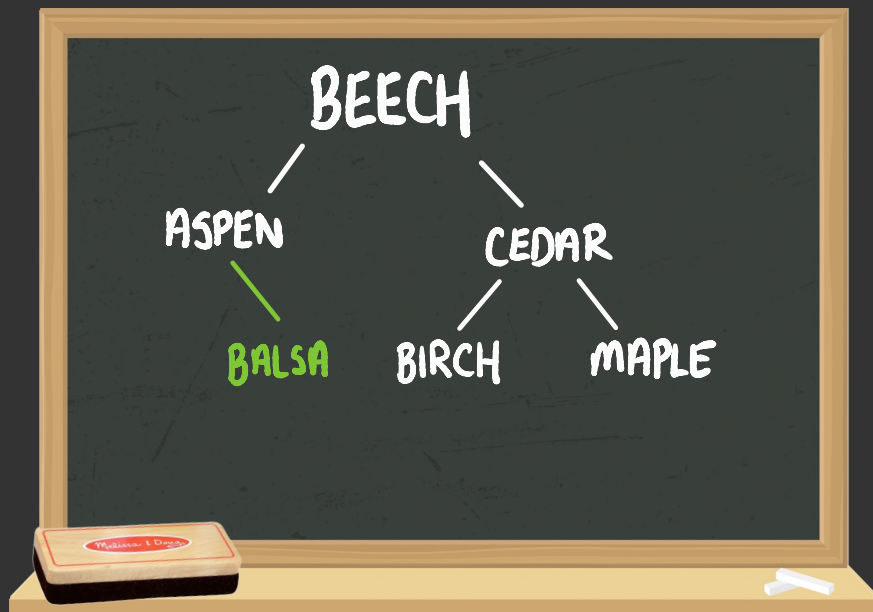
idea: organize the woods into a tree!



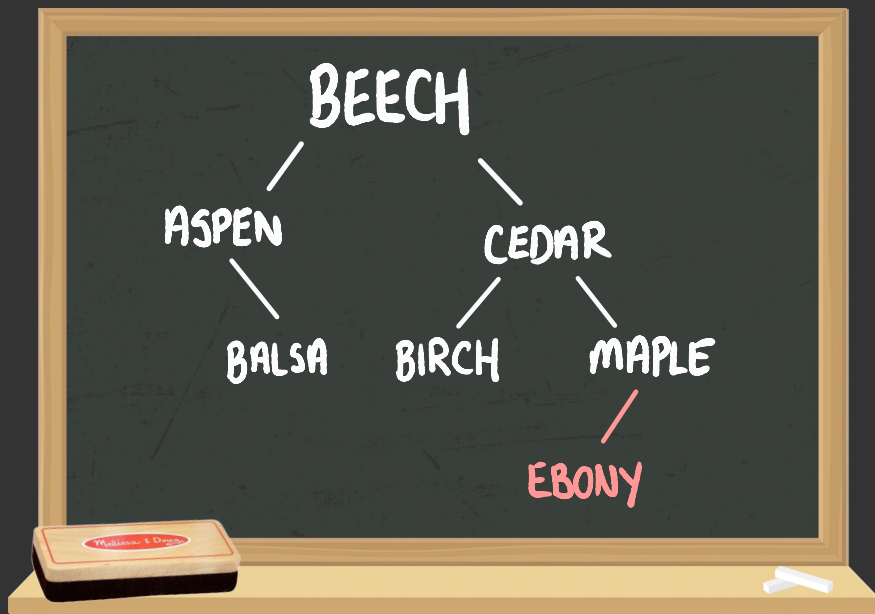
as field reports come back,
we put them in the tree
where they belong

by jove,
birch!

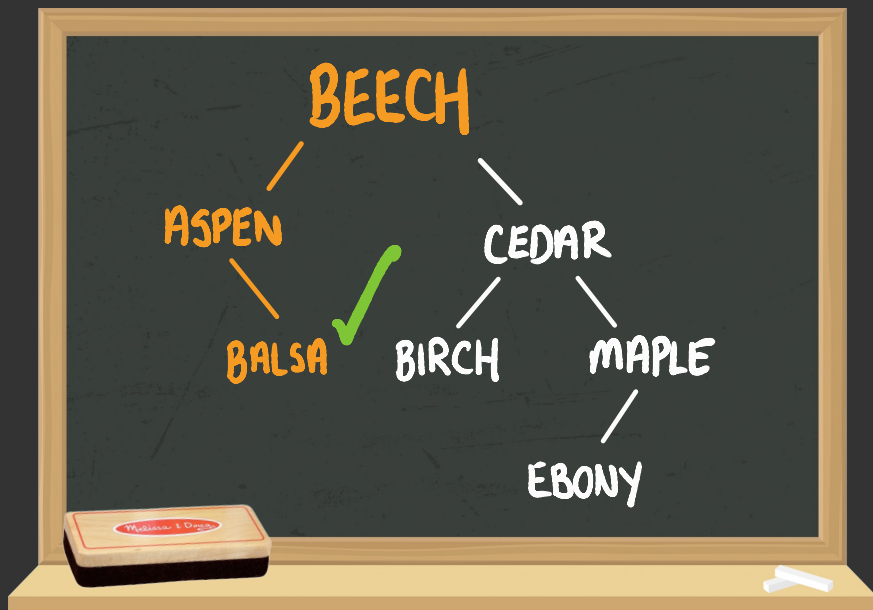




as field reports come back,
we put them in the tree
where they belong



as field reports come back,
we put them in the tree
where they belong



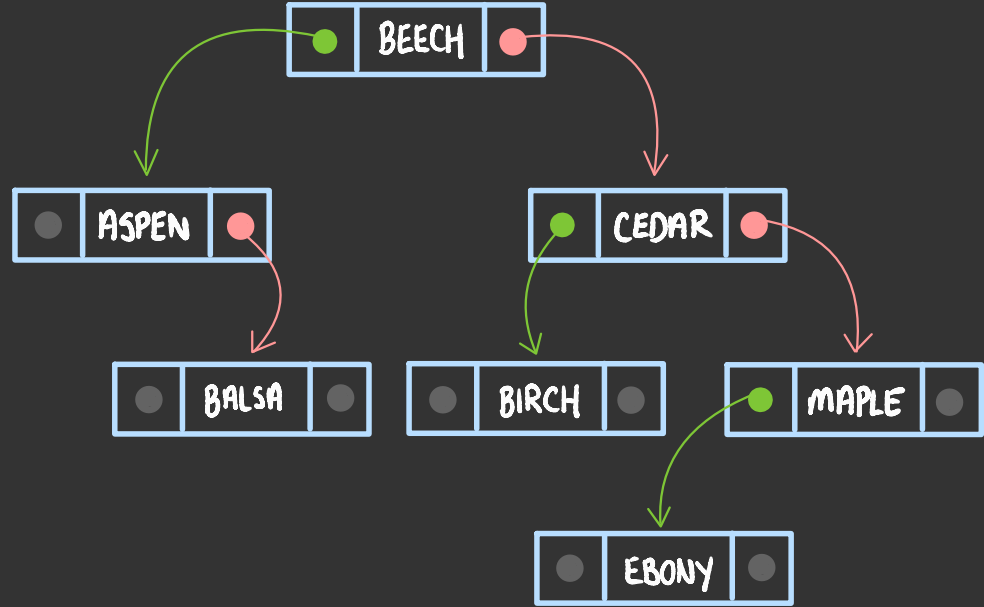
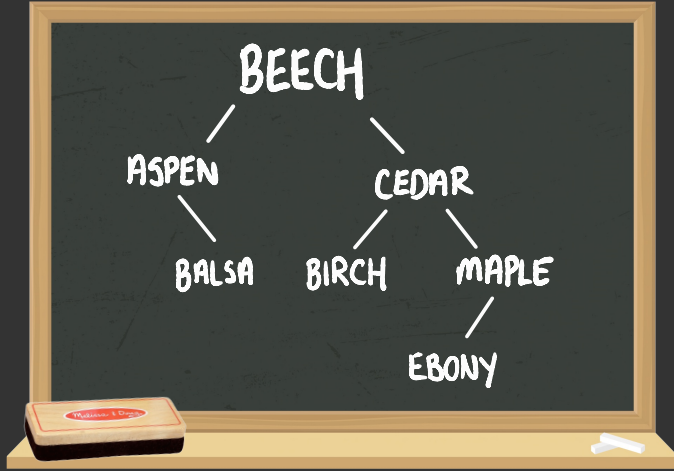
by jove,
balsa !



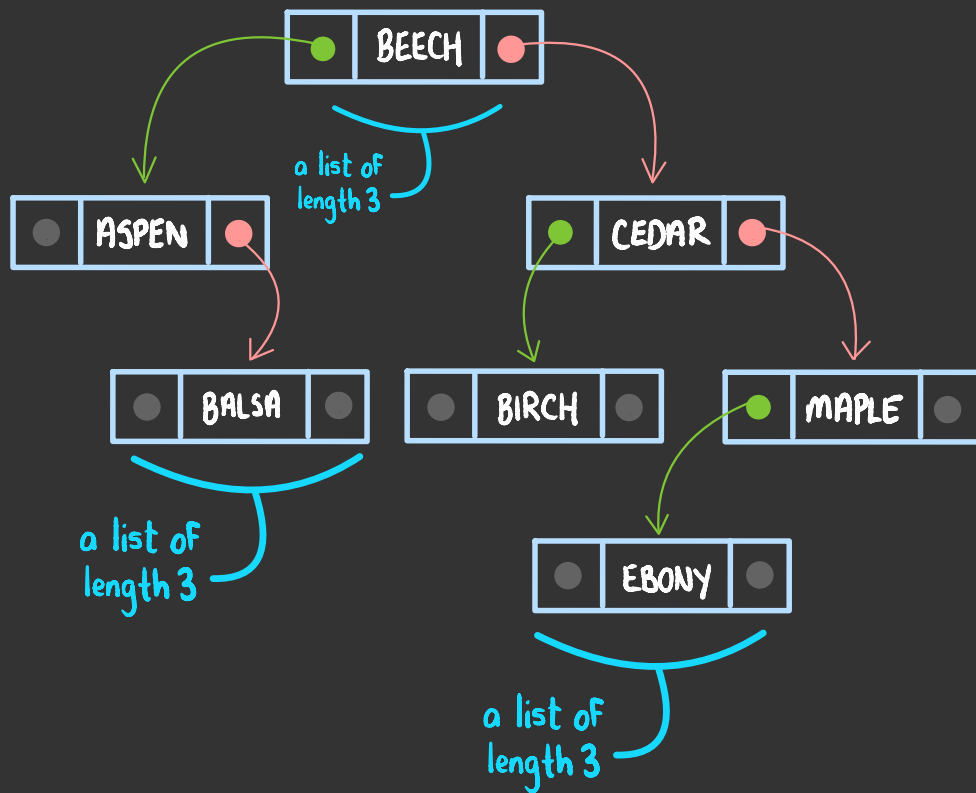
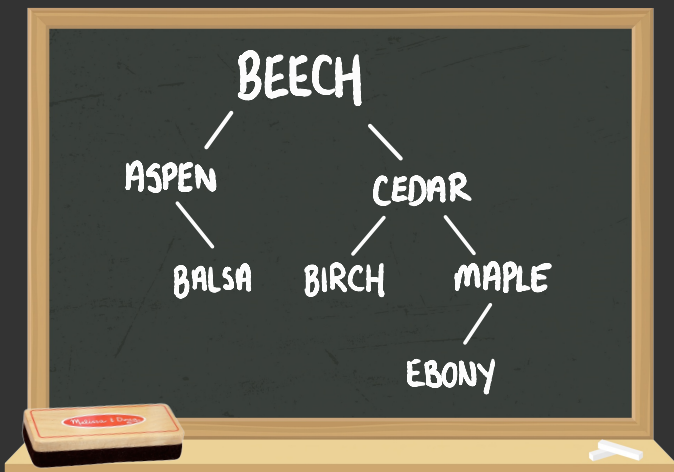
this is much less work, and allows us to quickly determine if a wood is already in our set

how might we do this
in python??



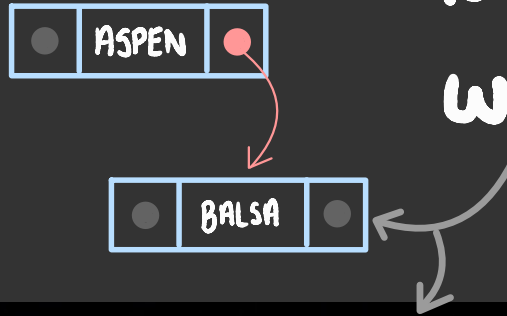


we can implement a tree using **lists** of length 3



we can implement a tree using **lists** of length 3

we'll represent the
lack of a child
with **None**



```
In [1]: balsa = [None, "BALSA", None]
```

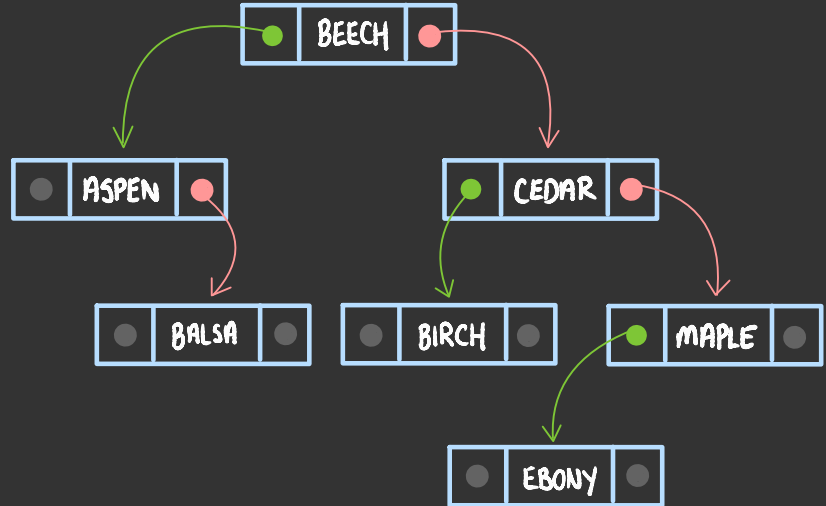
```
In [2]: aspen = [None, "ASPEN", balsa]
```

```
In [3]: aspen
```

```
Out[3]: [None, 'ASPEN', [None, 'BALSA', None]]
```

we can implement a tree using **lists** of length 3

```
In [1]: balsa = [None, "BALSA", None]
In [2]: aspen = [None, "ASPEN", balsa]
In [3]: birch = [None, "BIRCH", None]
In [4]: ebony = [None, "EBONY", None]
In [5]: maple = [ebony, "MAPLE", None]
In [6]: cedar = [birch, "CEDAR", maple]
In [7]: beech = [aspen, "BEECH", cedar]
```



a search algorithm

```
if wood < label and left != None:  
    tree = left
```



```
elif wood > label and right != None:  
    tree = right
```



```
else:  
    found_it = True
```

is **BALSA**
in the tree?

a search algorithm

```
def binary_search(wood, tree):  
    found_it = False  
    while not found_it:  
        left, label, right = tree  
        if wood < label and left != None:  
            tree = left  
        elif wood > label and right != None:  
            tree = right  
        else:  
            found_it = True  
    return tree
```

